

데이터베이스가 데드락을 처리하는 방식

Mysql InnoDB를 기반으로

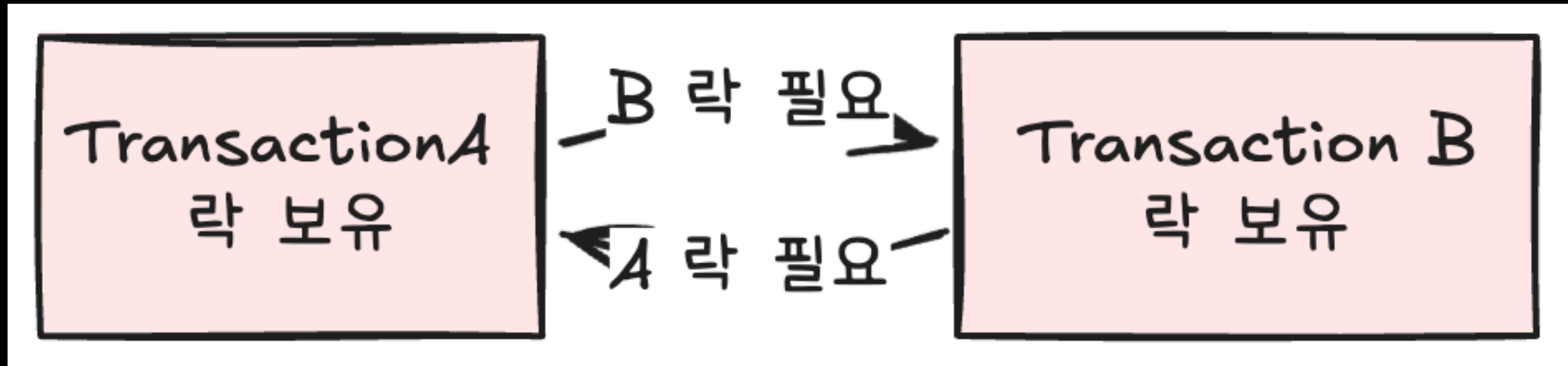
발표 순서

어떤 마음으로 발표를 들으면 좋을까?

1. 데드락이 무엇인가?
2. 부모 - 자식 테이블(FK)로 인한 락 전파
3. 데이터베이스가 데드락을 처리하는 방식(With Mysql InnoDB)

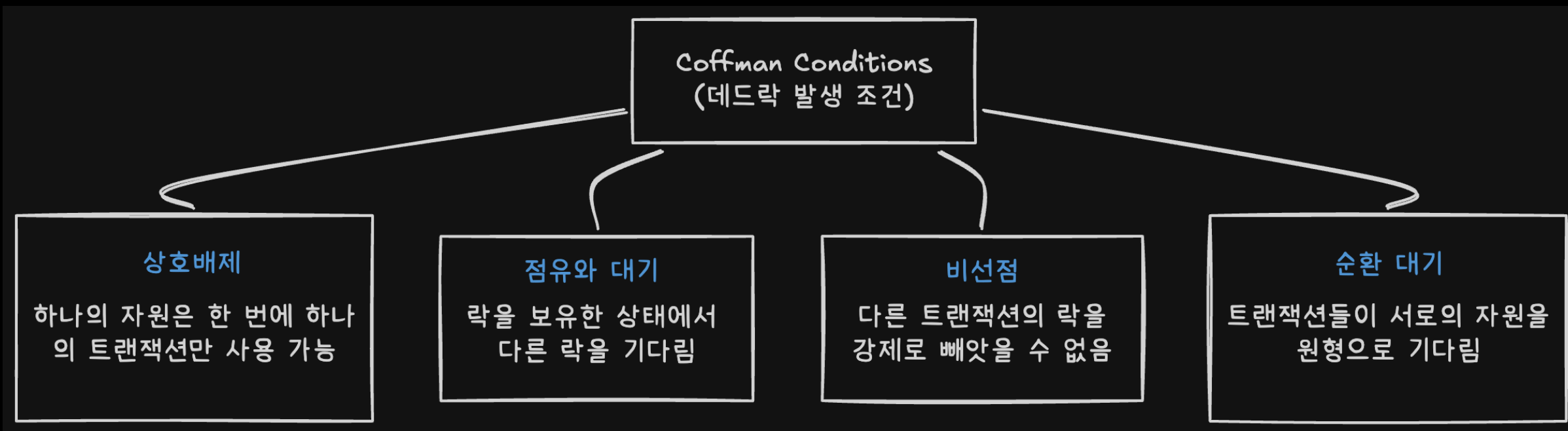
1. 데드락이란 무엇인가?

둘 이상의 트랜잭션이 서로 점유한 자원을 상대방이 해제해주기를 기다리며 무한히 대기하는 현상



2. 데드락이 발생하는 경우

이론적으로



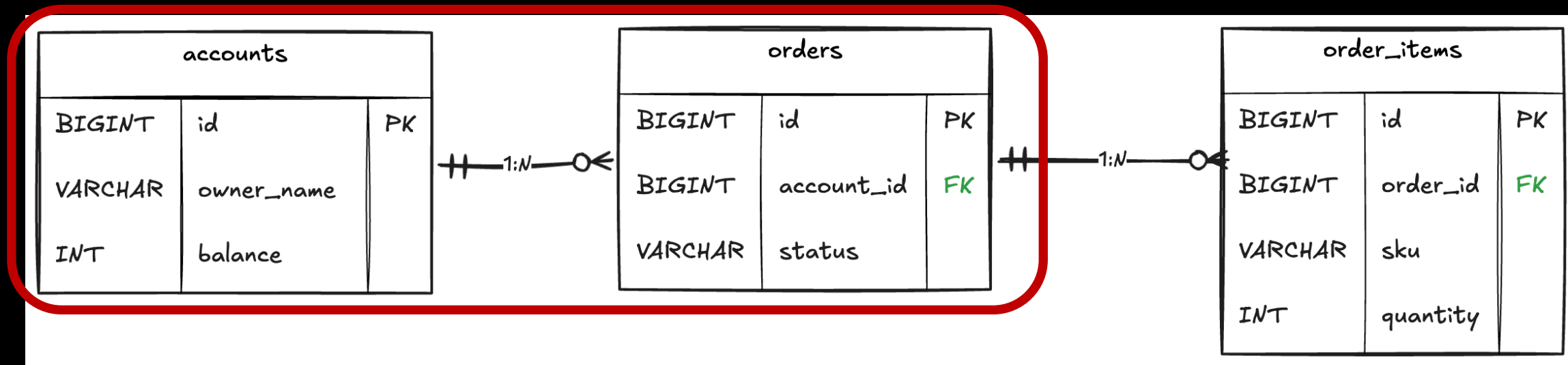
2. 데드락이 발생하는 경우

실제 환경에서

1. 여러 행을 서로 다른 순서로 업데이트
2. 부모 - 자식 테이블(FK)로 인한 잠금 전파
3. 인덱스 범위 조회(next-key lock)
4. INSERT와 UNIQUE 인덱스 충돌
5. 배치 작업과 일반 트랜잭션이 동시에 같은 데이터를 수정

3. 데드락 발생 케이스

부모 - 자식 테이블(FK)로 인한 잠금 전파 시



현재 테이블 구조: orders가 account_id를 참조하는 상황

3. 데드락 발생 케이스

부모 - 자식 테이블(FK)로 인한 잠금 전파 시

트랜잭션A가 accounts id = 1 레코드에 대한 X락 획득

UPDATE accounts SET balance = balance + 10000 WHERE id = 1;

트랜잭션B가 accounts id = 2 레코드에 대한 X락 획득

UPDATE accounts SET balance = balance + 10000 WHERE id = 2;

트랜잭션A가 account_id = 2를 갖는 orders의 새로운 레코드 생성

INSERT INTO orders (id, account_id, status) VALUES (101, 2, 'CREATED_BY_TX_A'); -- 이때, accounts의 id = 2 레코드에 대한 S락 획득 실패 후 대기

트랜잭션B가 account_id = 1를 갖는 orders의 새로운 레코드 생성

INSERT INTO orders (id, account_id, status) VALUES (102, 1, 'CREATED_BY_TX_A'); -- 이때, accounts의 id = 1 레코드에 대한 S락 획득 실패 후 대기

샤를의 트랜잭션(A)

accounts

id	name	balance
0	샤를	10,000
1	호날두	50,000
2		
3		
4		

orders

id	account_id	status
0		
1		
2		
3		

호날두의 트랜잭션(B)



샤를의 트랜잭션(A)

1. 샤를이 계좌에 1000원 추가

accounts

id	name	balance
0	샤를	10,000
1	호날두	50,000
2		
3		
4		

orders

id	account_id	status
0		
1		
2		
3		

호날두의 트랜잭션(B)



샤를의 트랜잭션(A)

1. 샤를이 계좌에 1000원 추가

트랜잭션 A
account id = 0에
x락 획득

accounts

id	name	balance
0	샤를	10,000
1	호날두	50,000
2		
3		
4		

orders

id	account_id	status
0		
1		
2		
3		

호날두의 트랜잭션(B)



샤를의 트랜잭션(A)

1. 샤를이 계좌에 1000원 추가

트랜잭션 A
account id = 0에
x락 획득

accounts

id	name	balance
0	샤를	10,000
1	호날두	50,000
2		
3		
4		

orders

id	account_id	status
0		
1		
2		
3		

호날두의 트랜잭션(B)

2. 호날두가 계좌에 1000원 추가



샤를의 트랜잭션(A)

1. 샤를이 계좌에 1000원 추가

트랜잭션 A
account id = 0에
X락 획득

accounts

id	name	balance
0	샤를	10,000
1	호날두	50,000
2		
3		
4		

트랜잭션 B
account id = 1에
X락 획득

호날두의 트랜잭션(B)

2. 호날두가 계좌에 1000원 추가

orders

id	account_id	status
0		
1		
2		
3		



샤를의 트랜잭션(A)

1. 샤를이 계좌에 1000원 추가
3. 호날두의 새로운 Order를 생성

트랜잭션 A
account id = 0에
x락 획득

accounts

id	name	balance
0	샤를	10,000
1	호날두	50,000
2		
3		
4		

orders

id	account_id	status
0		
1		
2		
3		

트랜잭션 B
account id = 1에
x락 획득

호날두의 트랜잭션(B)

2. 호날두가 계좌에 1000원 추가



샤를의 트랜잭션(A)

1. 샤를이 계좌에 1000원 추가
3. 호날두의 새로운 Order를 생성

트랜잭션 A
account id = 0에
X락 획득

accounts

id	name	balance
0	샤를	10,000
1	호날두	50,000
2		
3		
4		

account_id = 1,
status = Ready

orders

id	account_id	status
0	1	READY
1		
2		
3		

트랜잭션 B
account id = 1에
X락 획득

호날두의 트랜잭션(B)

2. 호날두가 계좌에 1000원 추가



샤를의 트랜잭션(A)

1. 샤를이 계좌에 1000원 추가
3. 호날두의 새로운 Order를 생성

트랜잭션 A
account id = 0에
X락 획득

accounts

id	name	balance
0	샤를	10,000
1	호날두	50,000
2		
3		
4		

account_id = 1,
status = Ready

orders

id	account_id	status
0	1	READY
1		
2		
3		

S락 획득 시도
실패 -> 트랜잭션 B가 끝날때까지 대기

트랜잭션 B
account id = 1에
X락 획득

호날두의 트랜잭션(B)

2. 호날두가 계좌에 1000원 추가



샤를의 트랜잭션(A)

1. 샤를이 계좌에 1000원 추가
3. 호날두의 새로운 Order를 생성

트랜잭션 A
account id = 0에
X락 획득

accounts

id	name	balance
0	샤를	10,000
1	호날두	50,000
2		
3		
4		

account_id = 1,
status = Ready

orders

id	account_id	status
0	1	READY
1	0	READY
2		
3		

트랜잭션 B
account id = 1에
X락 획득

호날두의 트랜잭션(B)

2. 호날두가 계좌에 1000원 추가
4. 샤를의 새로운 Order를 생성

account_id = 0,
status = Ready

X락 획득 시도
실패 -> 트랜잭션 B가 끝날때까지 대기



샤를의 트랜잭션(A)

1. 샤를이 계좌에 1000원 추가
3. 호날두의 새로운 Order를 생성

트랜잭션 A
account id = 0에
x락 획득

accounts

id	name	balance
0	샤를	10,000
1	호날두	50,000
2		
3		
4		

account_id = 1,
status = Ready

orders

id	account_id	status
0	1	READY
1	0	READY
2		
3		

트랜잭션 B
account id = 1에
x락 획득

호날두의 트랜잭션(B)

2. 호날두가 계좌에 1000원 추가
4. 샤를의 새로운 Order를 생성

account_id = 0,
status = Ready

S락 획득 시도

실패 -> 트랜잭션 B가 끝날때까지 대기

S락 획득 시도

실패 -> 트랜잭션 A가 끝날때까지 대기



샤를의 트랜잭션(A)

1. 샤를이 계좌에 1000원 추가
3. 호날두의 새로운 Order를 생성

트랜잭션 A
account id = 0에
X락 획득

accounts

id	name	balance
0	샤를	10,000
1	호날두	50,000
2		
3		
4		

account_id = 1,
status = Ready

orders

id	account_id	status
0	1	READY
1	0	READY
2		
3		

트랜잭션 B
account id = 1에
X락 획득

호날두의 트랜잭션(B)

2. 호날두가 계좌에 1000원 추가
4. 샤를의 새로운 Order를 생성

account_id = 0,
status = Ready

X락 획득 시도

실패 -> 트랜잭션 B가 끝날때까지 대기

X락 획득 시도

실패 -> 트랜잭션 A가 끝날때까지 대기



샤를의 트랜잭션(A)

1. 샤를이 계좌에 1000원 추가
3. 호날두의 새로운 Order를 생성

트랜잭션 A
account id = 0에
X락 획득

accounts

id	name	balance
0	샤를	10,000
1	호날두	50,000
2		
3		
4		

account_id = 1,
status = Ready

orders

id	account_id	status
0	1	READY
1	0	READY
2		
3		

해당 공유락은 개발자가 지정한 것이 아닌
DB가 FK 무결성 제약을 위해 자동으로 획득 시도한 락

즉, 예상치 못한 락 시도!

트랜잭션 B
account id = 1에
X락 획득

호날두의 트랜잭션(B)

2. 호날두가 계좌에 1000원 추가
4. 샤를의 새로운 Order를 생성

account_id = 0,
status = Ready

락 획득 시도

실패 -> 트랜잭션 B가 끝날때까지 대기

락 획득 시도

실패 -> 트랜잭션 A가 끝날때까지 대기

3. 데드락 발생 케이스

부모 - 자식 테이블(FK)로 인한 잠금 전파 시

모든 데드락 발생 가능성을 코드 수준에서
완전히 예측하고 제거하기는 어렵다.

3. 데드락 발생 케이스

부모 - 자식 테이블(FK)로 인한 잠금 전파 시

해당 문제 재연 테스트코드 바로가기



<https://github.com/sangjun121/deadlock-detection-playground>

4. 데드락이 발생한다면,

데드락이 발생하는 경우, DB는 먹통이 될까?

운영 중에 데드락이 발생한다면,

바로 DB에 장애가 일어나고 멈춰버리게 될까?

4. 데드락이 발생한다면, 발생할 문제점

데드락이 발생하는 경우, DB는 먹통이 될까? **그렇지 않다.**

멀티 트랜잭션 환경에서 데드락이 걸린다는 것은,
어플리케이션이 정지됨을 의미한다.

4. 데드락이 발생한다면, 발생할 문제점

데드락이 발생하는 경우, DB는 먹통이 될까? 그렇지 않다.

데드락에 정지된 요청에 대해,
사용자는 5초 이상 대기한 경우 재요청 시도

4. 데드락이 발생한다면, 발생할 문제점

데드락이 발생하는 경우, DB는 먹통이 될까? **그렇지 않다.**

결국 같은 이유로, 같은 레코드에 데드락 발생



4. 데드락이 발생한다면, 발생할 문제점

데드락이 발생하는 경우, DB는 먹통이 될까? **그렇지 않다.**

결국 같은 이유로, 같은 레코드에 데드락 발생

결국, DB 커넥션을 잡아두어, 풀이 줄어드는 문제도 발생할 수 있다.

5. 데이터베이스가 데드락을 해결하는 방식

그렇기 때문에 데이터베이스는 기본적으로
데드락을 탐지하는 기능을 제공한다.

5. 데이터베이스가 데드락을 해결하는 방식

Mysql의 스토리지 엔진 InnoDB를 기반으로

InnoDB의 데드락 탐지 스레드

특정 시점에 데드락이 발생했는지를 그래프를 순회하여 감지하고,
관련 트랜잭션 중 하나를 희생자로 선택해 롤백시키는 작업으로 데드락 해제.

5. 데이터베이스가 데드락을 해결하는 방식

1. 데드락을 탐지하는 주체

InnoDB의 데드락 탐지 스레드: 주체

특정 시점에 데드락이 발생했는지를 그래프를 순회하여 감지하고,
관련 트랜잭션 중 하나를 희생자로 선택해 롤백시키는 작업으로 데드락 해제.

5. 데이터베이스가 데드락을 해결하는 방식

1. 데드락을 탐지하는 시점

InnoDB의 데드락 탐지 스레드

특정 시점에 데드락이 발생했는지를 그래프를 순회하여 감지하고,
관련 트랜잭션 중 하나를 희생자로 선택해 롤백시키는 작업으로 데드락 해제.

트랜잭션이 락 경합으로 인해 대기상태에 진입하는 시점

5. 데이터베이스가 데드락을 해결하는 방식

2. 데드락을 탐지하는 방식

InnoDB의 데드락 탐지 스레드

특정 시점에 데드락이 발생했는지를 **그래프를 순회하여** 감지하고,
관련 트랜잭션 중 하나를 희생자로 선택해 롤백시키는 작업으로 데드락 해제.

**내부적으로 대기 트랜잭션을 표현하는
Wait for List 그래프 관리**

5. 데이터베이스가 데드락을 해결하는 방식

2. 데드락을 탐지하는 방식

InnoDB의 데드락 탐지 스레드

특정 시점에 데드락이 발생했는지를 **그래프를 순회하여** 감지하고,
관련 트랜잭션 중 하나를 희생자로 선택해 롤백시키는 작업으로 데드락 해제.

이때, 순환 관계를 갖는 경우 데드락으로 판단

5. 데이터베이스가 데드락을 해결하는 방식

3. 희생자를 선택하는 방식

InnoDB의 데드락 탐지 스레드

특정 시점에 데드락이 발생했는지를 그래프를 순회하여 감지하고,
관련 트랜잭션 중 **하나를 희생자로 선택해** 롤백시키는 작업으로 데드락 해제.

Undo Log가 제일 적은 트랜잭션을 실패 처리하고 롤백

6. 자동 데드락 감지의 문제점

자동 데드락 옵션 `innodb_deadlock_detect`은 ON이 Default이다.

높은 동시성 환경에서 많은 트랜잭션이 동일한 락을 기다리는 경우,
데드락 탐지 스레드가 느려진다

6. 자동 데드락 감지의 문제점

자동 데드락 옵션 `innodb_deadlock_detect`은 ON이 Default이다.

데드락 감지 스레드도 락 목록을 조회하기 위해 **그래프에 새로운 락을 걸고** 탐지 작업을 수행하기 때문이다. 결국 성능 저하를 유발할 수 있다.

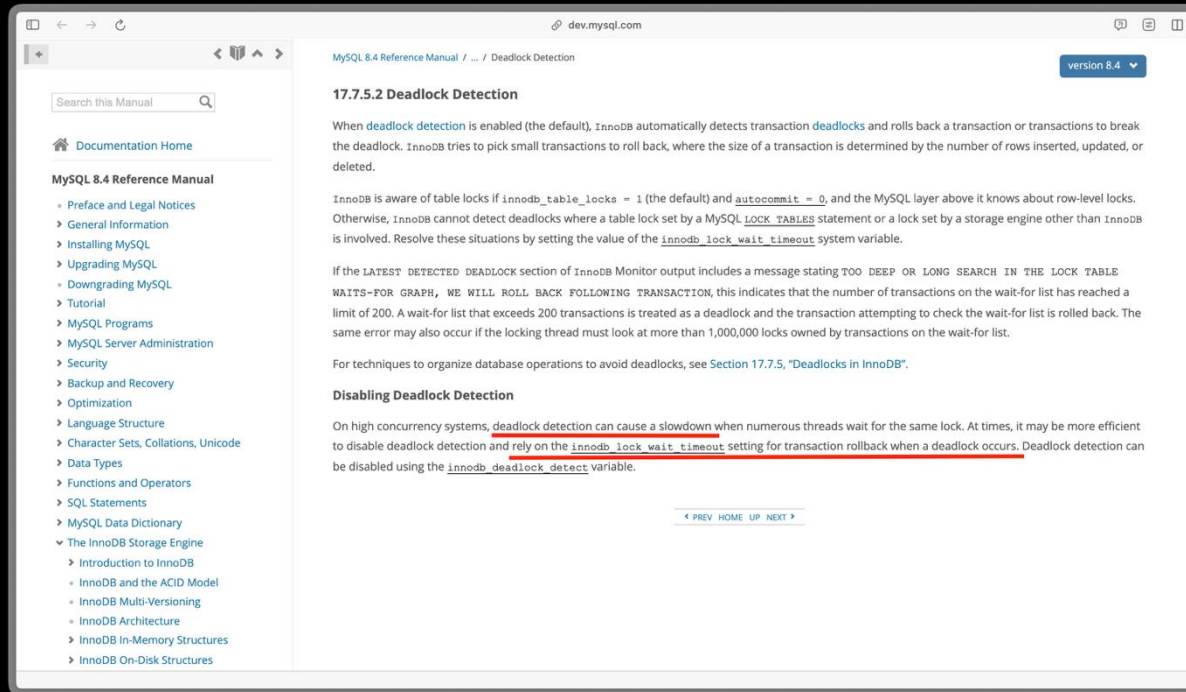
6. 자동 데드락 감지의 문제점

자동 데드락 옵션을 끄고, `innodb_lock_wait_timeout`에 의존하기

`innodb_lock_wait_timeout` 값에 의존하여
락 대기 시간이 초과되었을 때
트랜잭션을 롤백하는 것이 더 효율적일 수 있다.

6. 자동 데드락 감지의 문제점

실제, MYSQL의 공식문서에서 제안하는 해결방식



<https://dev.mysql.com/doc/refman/8.4/en/innodb-deadlock-detection.html>

7. 발표내용을 블로그로 읽어보기



<https://junsangcho.tistory.com/26>